

Model merging

Agenda

- Motivation and idea
- Merging overview

- Pre-merging methods
- During-Merging Methods
- Merging for LLMs

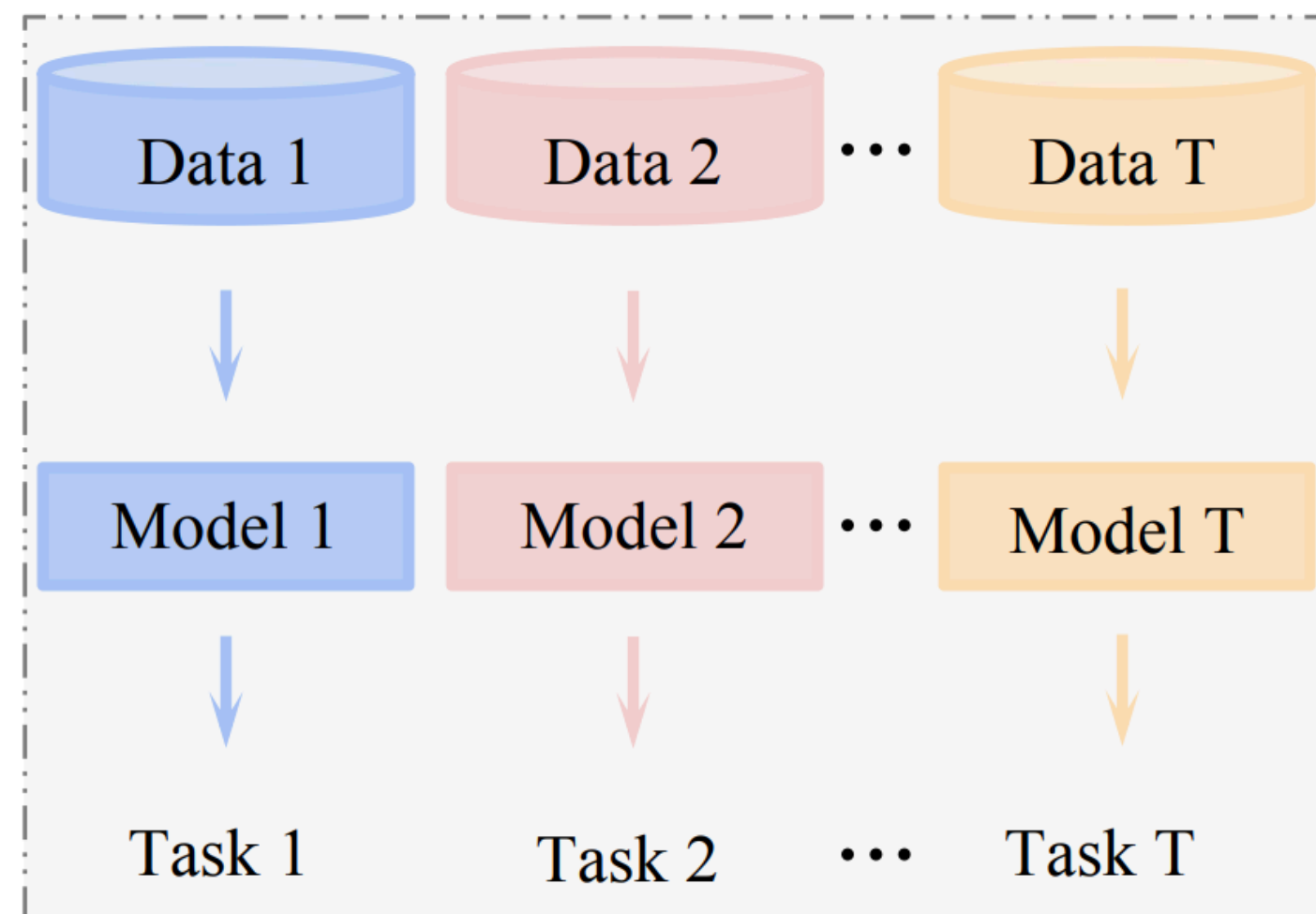
Motivation

Suppose we fine-tuned a model

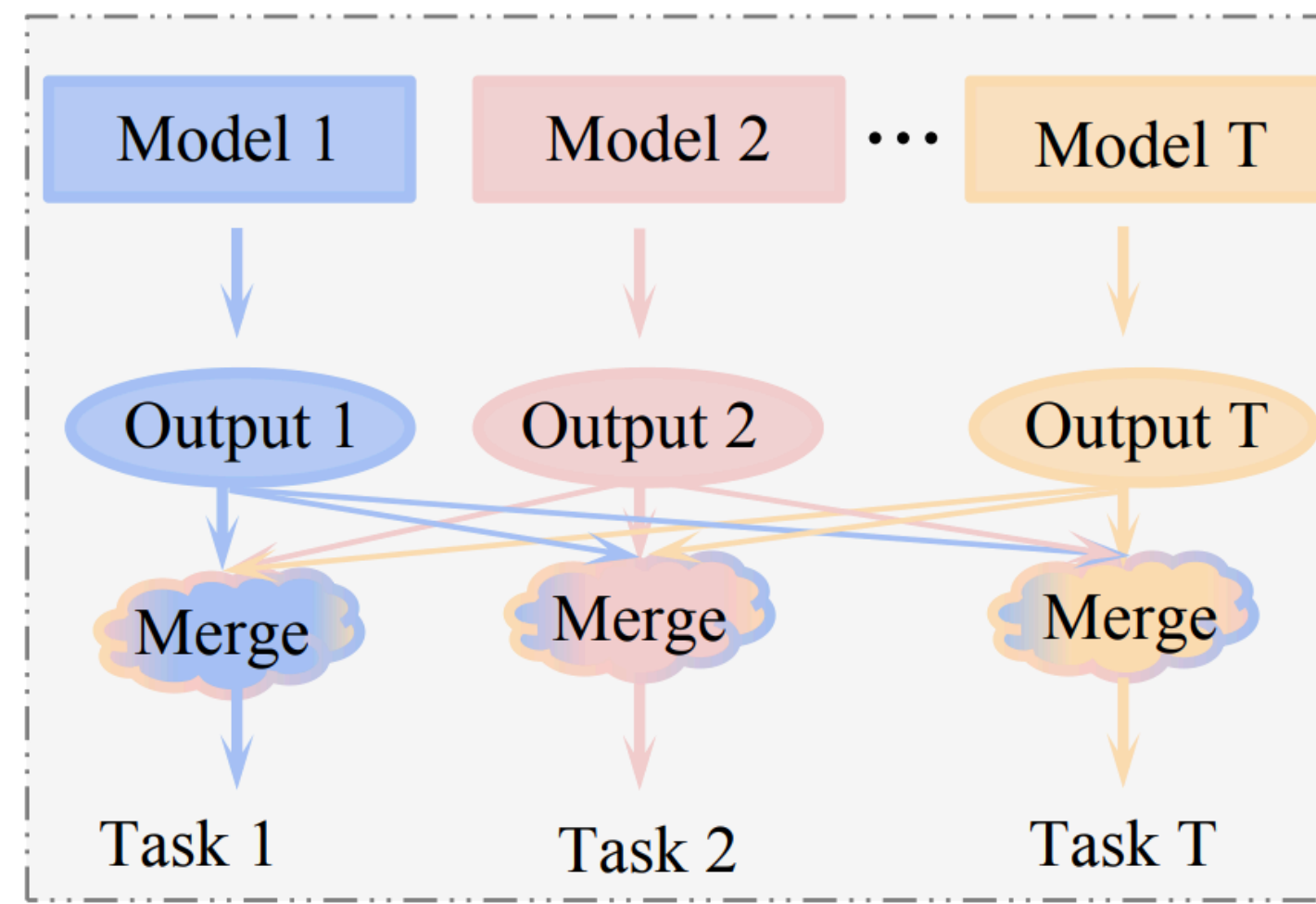
- What if model forgets some general knowledge?
- What if pre-trained model improved?
- What if we want to combine two models for different downstream tasks?

Merging allows to solve all this without additional training

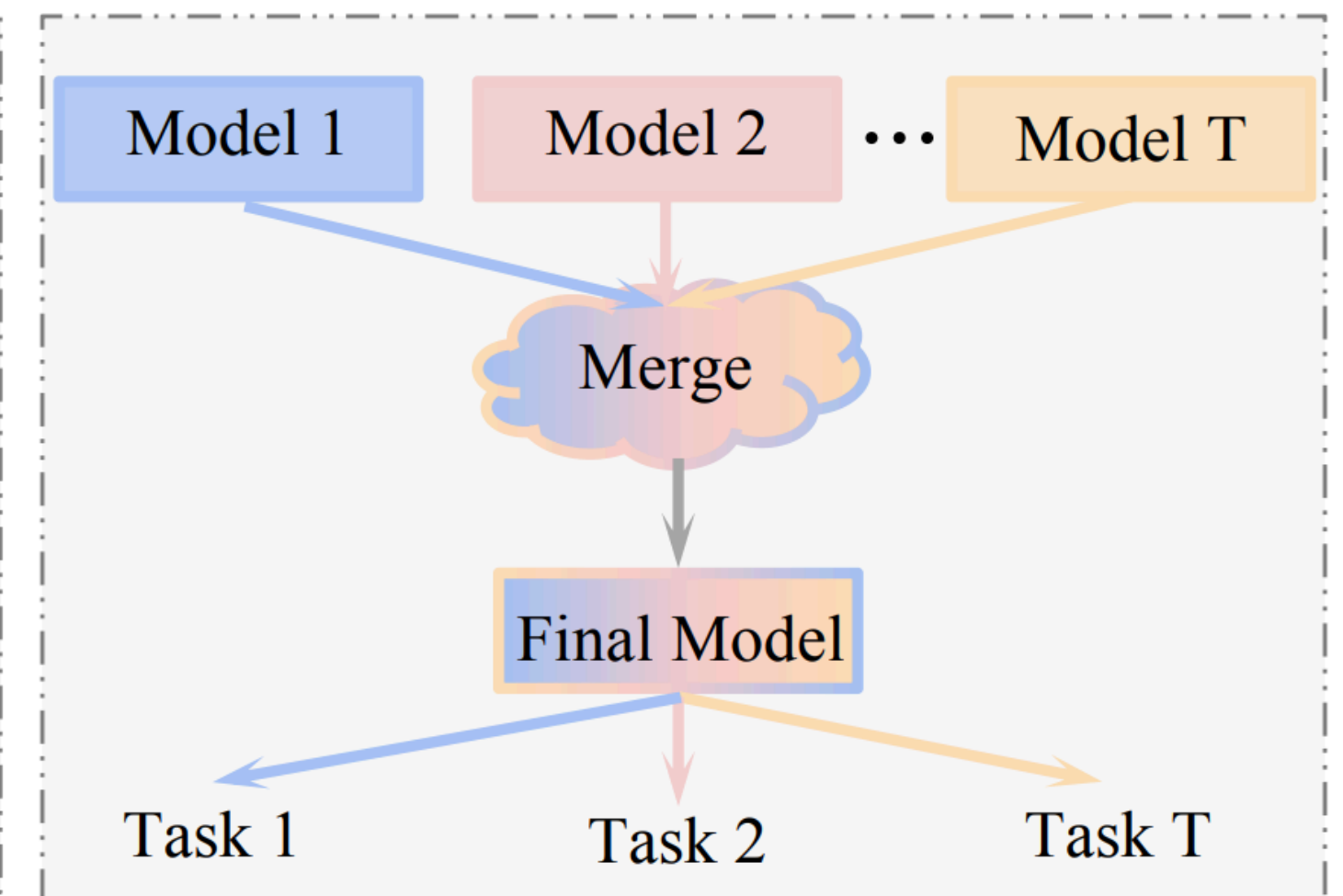
Merging approach



(a) Separate Models



(b) Ensemble Learning



(c) Model Merging

Pre-merging methods

The stage that prepares models for merging

- Linearization Fine-tuning
- Architecture Transformation
- Weight Alignment

Linearization Fine-tuning

Task arithmetic: Let $\tau_1 = W_{FT_1} - W_{PT}$ and $\tau_2 = W_{FT_2} - W_{PT}$

Then, $W_{PT} + \tau_1 + \tau_2$ — weights of a model that can solve task1 and task2

Linearization Fine-tuning

Task arithmetic: Let $\tau_1 = W_{FT_1} - W_{PT}$ and $\tau_2 = W_{FT_2} - W_{PT}$

Then, $W_{PT} + \tau_1 + \tau_2$ — weights of a model that can solve task1 and task2

Neural Tangent Kernel (NTK):

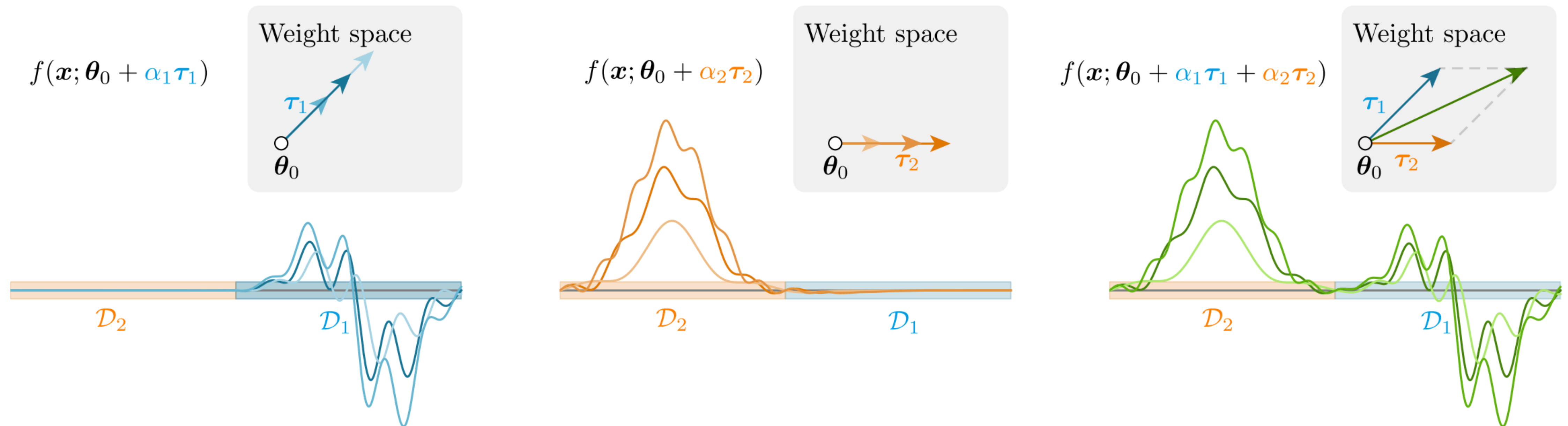
$$f(x, \theta) \approx f(x, \theta_0) + (\theta - \theta_0)^T \nabla_{\theta} f(x, \theta_0)$$

- During **pre-training** this equation is **incorrect**
- During **fine-tuning** it is **correct**, because weight's change is minimal
=> the relationship between weights and a function is **linear**

Linearization Fine-tuning

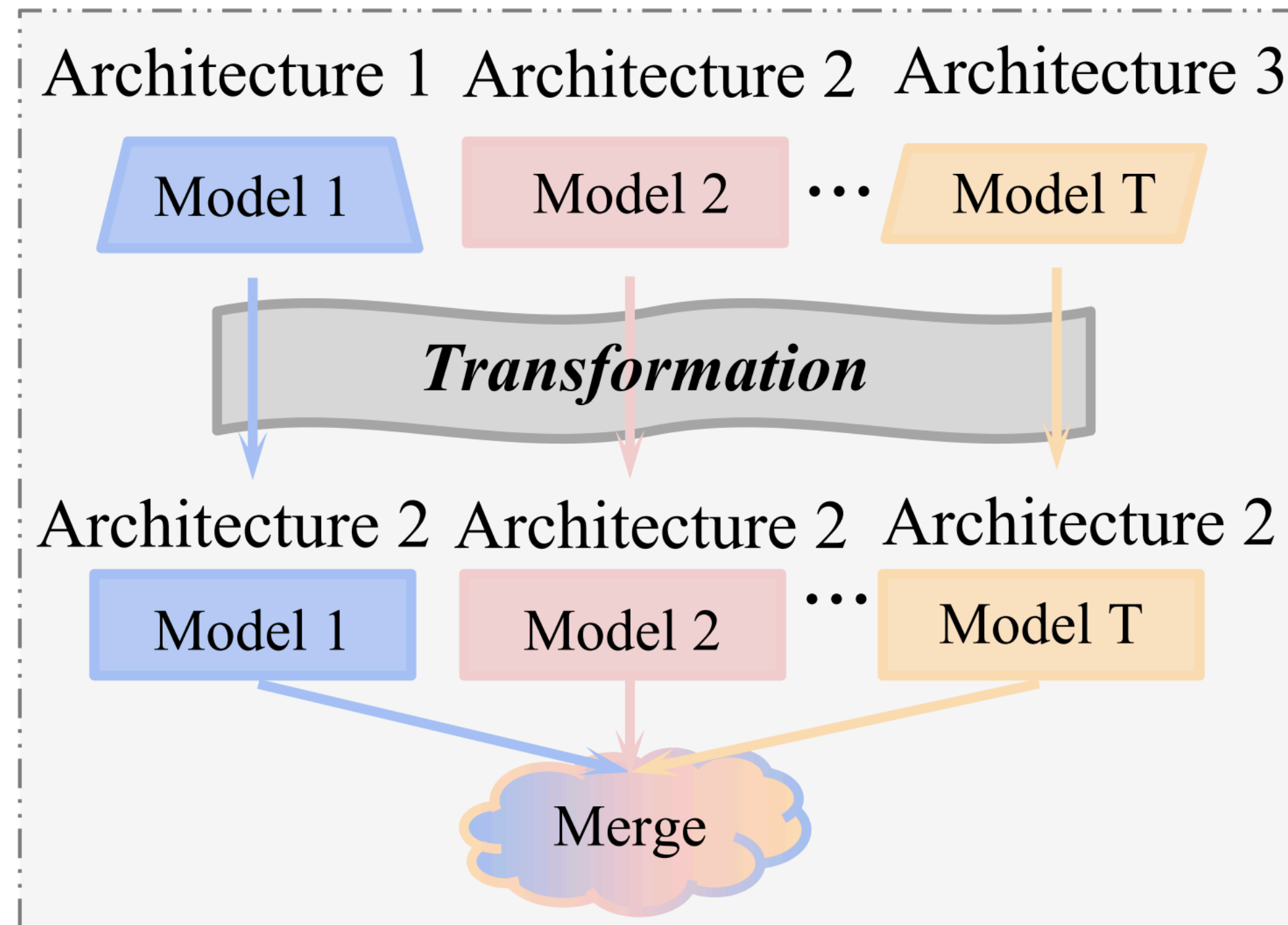
Turns out that task arithmetic works better when working with linearized models

$$f_{lin}(x, \theta + \tau) = f(x, \theta_0) + \tau^T \nabla_{\theta} f(x, \theta_0)$$



Architecture Transformation

1. Match the architectures of all models
2. Merge the resulting models



Architecture Transformation

Let $\{f_1, \dots, f_N\}$ be a set of models with different architectures each trained to a specific task.

Rooting (aligning architectures)

- Choose f_r as a root model
- For $j \in \{1, \dots, N\} \setminus r$, train f_r on predictions of f_j , obtaining $f_{r \rightarrow j}$

Architecture Transformation

Let $\{f_1, \dots, f_N\}$ be a set of models with different architectures each trained to a specific task.

Rooting (aligning architectures)

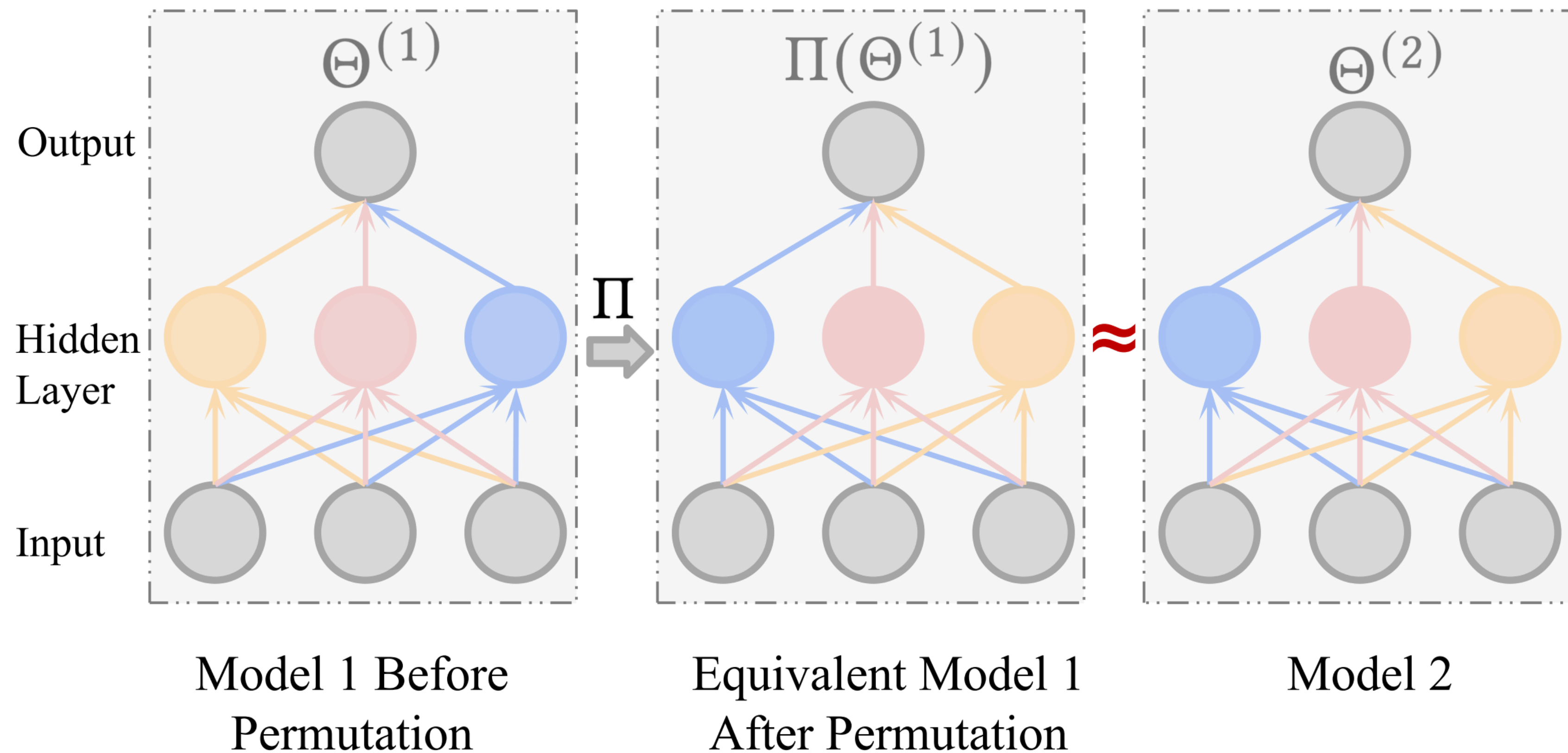
- Choose f_r as a root model
- For $j \in \{1, \dots, N\} \setminus r$, train f_r on predictions of f_j , obtaining $f_{r \rightarrow j}$

Merging

- Average the weights: $\theta = \frac{1}{N} \left(\theta_r + \sum_{j \in [N] \setminus r} \theta_{r \rightarrow j} \right)$
- Fine-tune $f(\theta)$ on outputs of $\{f_1, \dots, f_N\}$

Weight Alignment

The linear mode connectivity (LMC) suggests that all local minimal are connected. Using this property, align weights for easier merging.



Weight Alignment

There are several ways of aligning the weights:

1) Finding a permutation that maximizes the correlation between activations

$$P_l^* = \arg \min_{P_l \in \Pi_{K_l}} \sum_{i \in K_l} c(\mathbf{X}_{l,i}^{(1)}, \mathbf{X}_{l,P_l(i)}^{(2)}) \quad c(\mathbf{x}, \mathbf{y}) := 1 - \text{correlation}(\mathbf{x}, \mathbf{y})$$

Weight Alignment

There are several ways of aligning the weights:

1) Finding a permutation that maximizes the correlation between activations

$$P_l^* = \arg \min_{P_l \in \Pi_{K_l}} \sum_{i \in K_l} c(\mathbf{X}_{l,i}^{(1)}, \mathbf{X}_{l,P_l(i)}^{(2)}) \quad c(\mathbf{x}, \mathbf{y}) := 1 - \text{correlation}(\mathbf{x}, \mathbf{y})$$

2) Solving an optimal transport problem to find a permutation that aligns

$$\mathbf{X}_k[\ell] \leftarrow \widehat{\mathbf{W}}_k^{(\ell, \ell-1)}, \mathbf{Y}[\ell] \leftarrow \widehat{\mathbf{W}}_{\mathcal{F}}^{(\ell, \ell-1)} \quad \text{or} \quad \mathbf{X}_k \leftarrow \text{ACTS}(M_k(S)), \mathbf{Y} \leftarrow \hat{\text{ACTS}}(M_{\mathcal{F}}(S))$$

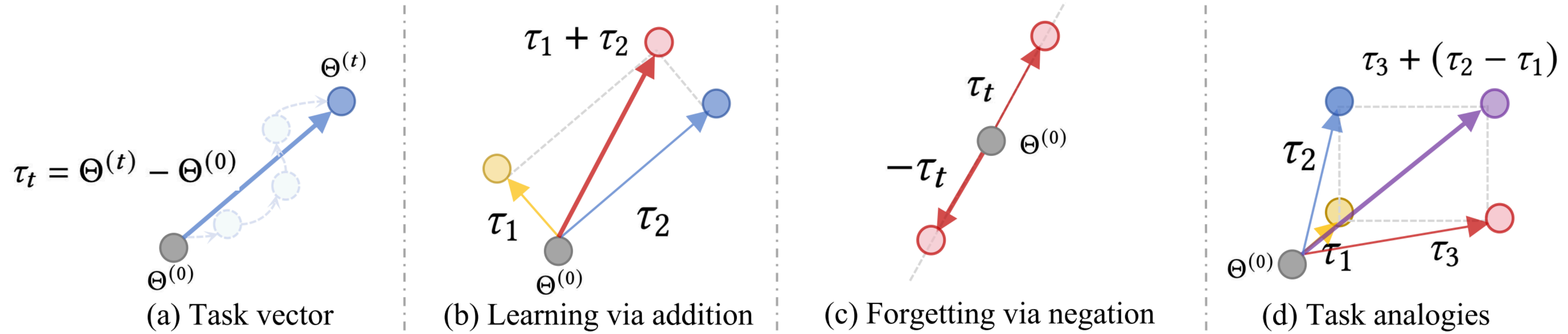

weight matrices


activations of a batch of data

During-Merging Methods

- Weighted-based Merging Methods
- Subspace-based Merging Methods
- Routing-based Merging Methods
- Theories and Analysis
- Post-calibration based Methods

Weighted-based Merging Methods



$$\theta_{merge} = \theta_0 + \lambda_1 \tau_1 + \lambda_2 \tau_2$$

How to find λ_1 and λ_2 ?

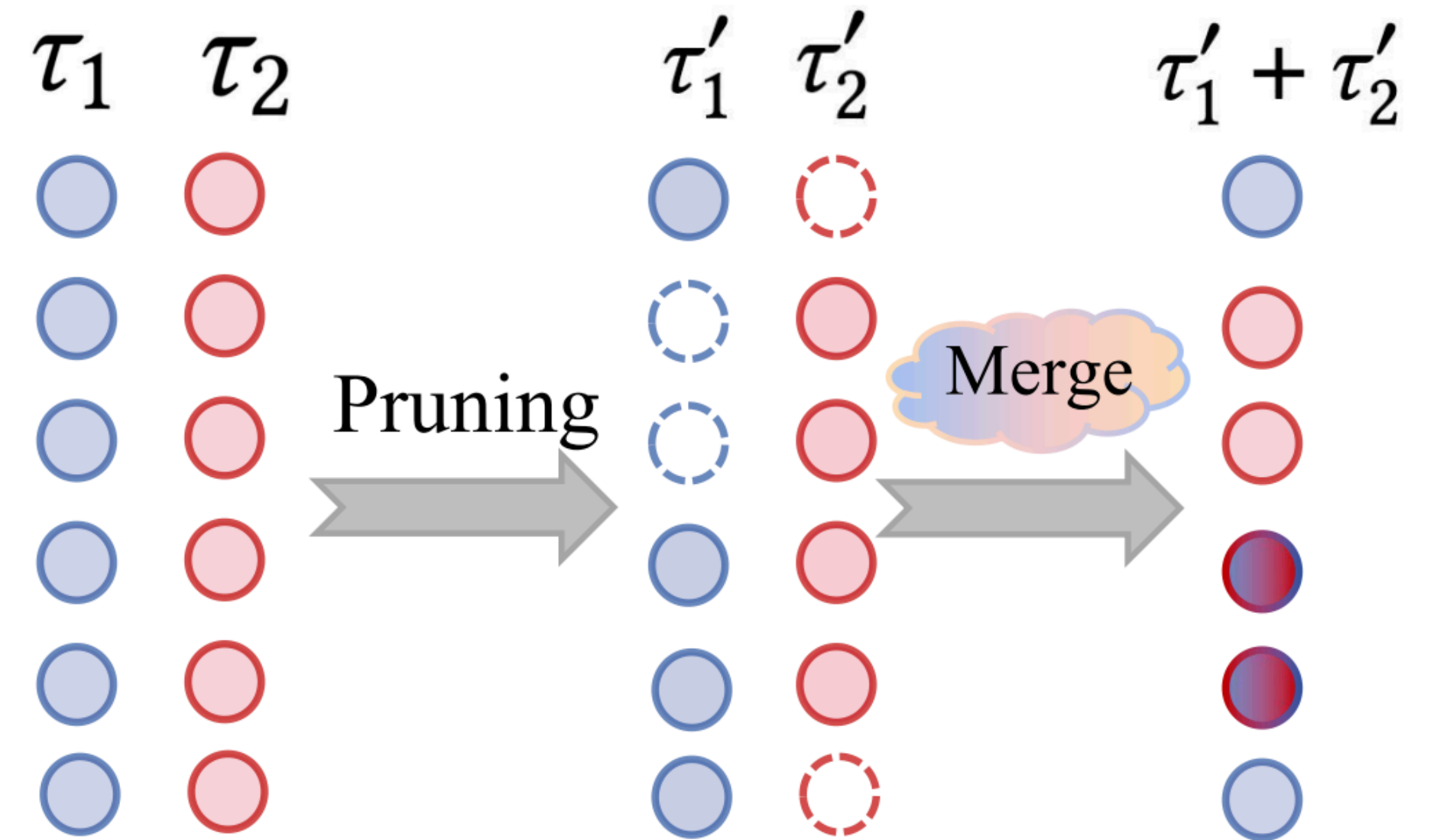
- 1) Set $\lambda_1 = \lambda_2 = 0.5$
- 2) Optimize them to get best multitask quality/entropy on unlabelled data
 - Evolutionary algorithms [Akiba et al, 2024]
 - Bayesian optimization [Liu et al, 2024]
 - Gradient descent [Yang et al, 2023]

Subspace-based Merging Methods

1) Prune the model or mask some weights based on

- Magnitude
- Influence on task quality
- Optimal configuration from learning

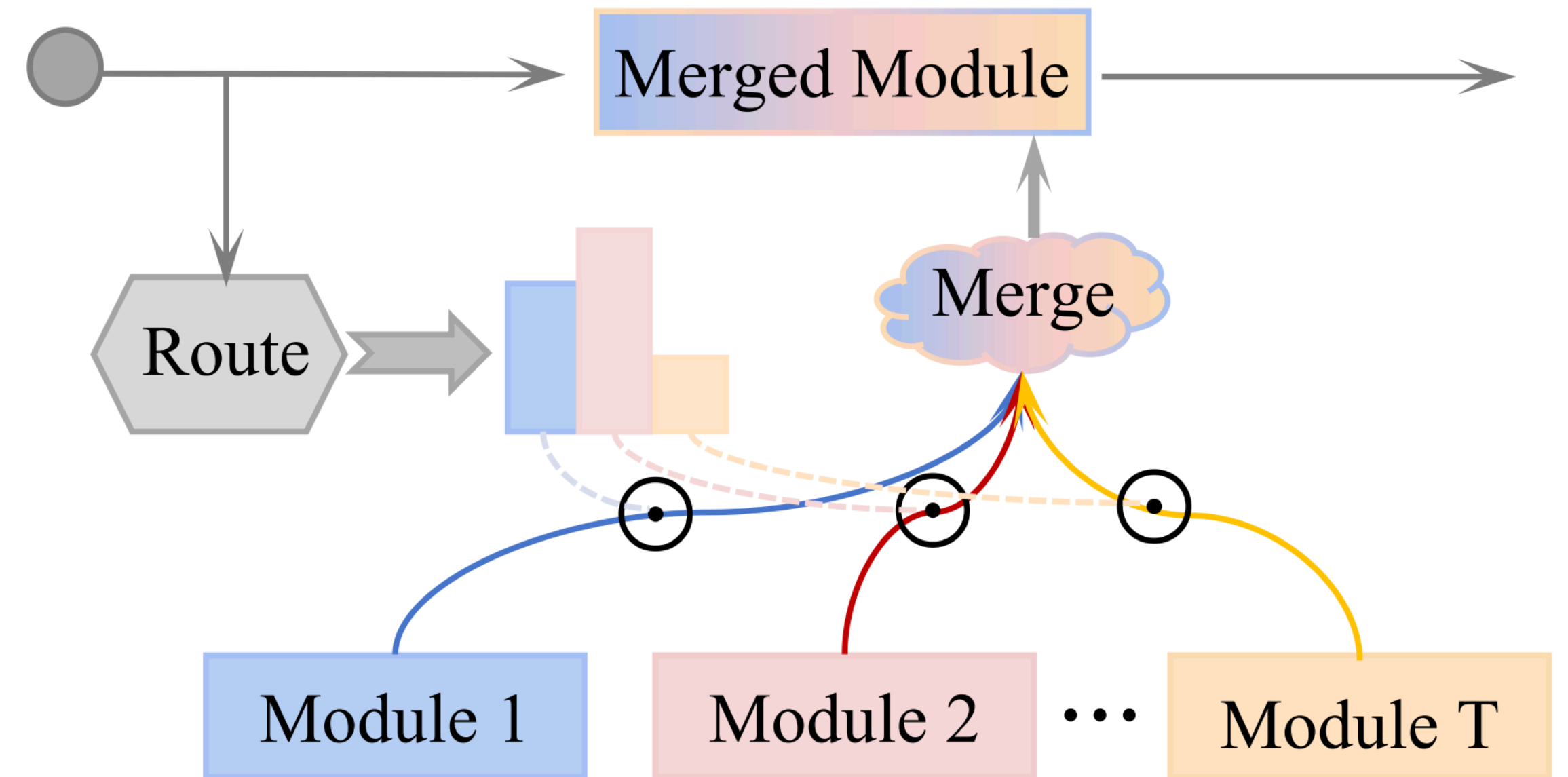
2) Merge models with task arithmetic



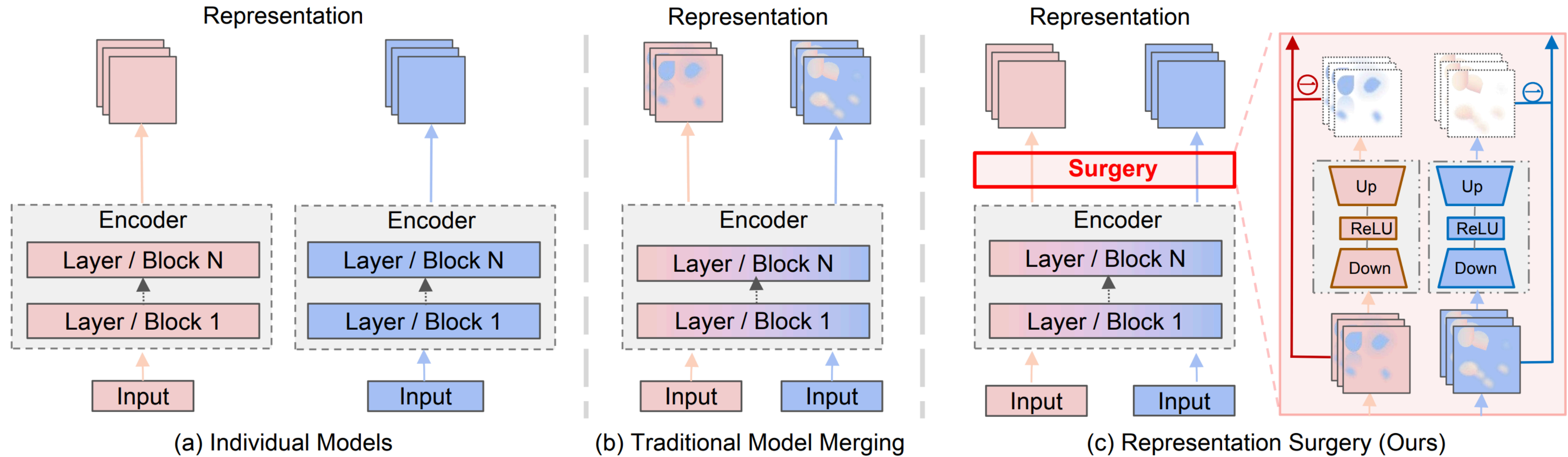
Routing-based Merging Methods

Change merged model depending on input/task.

Merge experts in MoE with coefficients obtained from router.



Post-calibration based Methods



The representations extracted with merged model differs a lot from representations of independent models

Calibrate merged model by aligning its representations with independent models

$$\arg \min_{\{\theta_{\Phi_1}, \theta_{\Phi_2}, \dots, \theta_{\Phi_T}\}} \sum_{t=1}^T \frac{1}{|D_{te}^t|} \|\hat{\mathcal{Z}}_t^{mtl} - \mathcal{Z}_t^{ind}\|_1$$

$$\text{s.t. } \hat{\mathcal{Z}}_t^{mtl} = \mathcal{Z}_t^{mtl} - \Phi_t(\mathcal{Z}_t^{mtl}),$$

$$\Phi_t(\mathcal{Z}_t^{mtl}) = \mathbf{W}_{up} \cdot \text{ReLU}(\mathbf{W}_{down} \cdot \mathcal{Z}_t^{mtl^\top})$$

Merging for LLMs

Human Preference Alignment

- Add task vector from a model fine-tuned on a small amount of preference data
- Merge Reward Model with LLMs fine-tuned on different domains
- Merge models trained on different preferences

Detoxification

- Detoxification by subtracting toxic task vector

Forgetting

- Removing private/copyrighted data from learned model (unlearning)